

STAT 4710/5710: Homework 1

David Dai

Due: September 16, 2025 at 11:59am

Contents

Instructions	1
1 R basics (10 points)	3
1.1 Creating vectors of divisible integers (5 points)	3
1.2 Creating repeated vectors (5 points)	3
Case study: NYC Flights 2013	4
2 Processing (45 points)	4
2.1 Import (5 points)	4
2.2 Tidy: missing values (5 points)	5
2.3 Tidy: filtering major carriers (15 points)	6
2.4 Quality control (20 points)	7
3 Explore (45 points)	10
3.1 Departure and arrival delays (15 points)	10
3.2 Delays by month and by carrier (20 points)	11

Instructions

Materials and collaboration

The policy on allowed materials and collaboration is as stated on the Syllabus:

“Students are permitted to work together on homework assignments, but must write up and submit solutions individually. In particular, students may not copy each others’ solutions. Students may consult all course materials, textbooks, the internet, or AI tools (e.g. ChatGPT) to complete their homework. Students may not use solutions to problems that may be available online and/or from past iterations of the course. For each homework, students must disclose all classmates with whom they collaborated, which AI tools they used, and how they used them. Failure to do so will result in a 10-point penalty.”

In accordance with this policy, please type in the answers after each:

Please disclose all classmates with whom you collaborated: Michael Fuentes

Please disclose which AI tools you used, and how you used them: ChatGPT for providing me examples in ggplot and summarize()

Failure to answer the above questions will result in a 10-point penalty.

Writeup

Use this document as a starting point for your writeup, adding your solutions after *****Solution***** for the corresponding questions. Add your R code using code chunks (with properly loaded packages) and add your text answers using ****bold text****. Consult the [preparing reports guide](#) for guidance on compilation, creation of figures and tables, and presentation quality. In particular, if the instructions ask you to “print a table”, you should use **kable**. If the instructions ask you to “print a tibble”, you should not use **kable** and instead print the tibble directly.

Programming

Apart from the “R basics” part, the **tidyverse** paradigm for data visualization, manipulation, and wrangling is **required**. Codes written in base R may receive deductions at the discretion of the instructor, since the purpose of assignments is to practice tidyverse workflows (e.g., **filter**, **mutate**, **group_by**, **summarize**, **join**, **ggplot2**).

Students who strongly prefer to use Python for homework may do so. If you choose this option, you must:

- Use **pandas** (or similar) for data wrangling with method chaining (**groupby**, **merge**, **assign**, etc.) rather than raw loops.
- Use **plotnine** for visualization (preferred, since it parallels ggplot2). If using seaborn/matplotlib, you must still include the required plot elements (labels, 45° line, smoothing curve, facets, etc.).
- Present both code chunks and the corresponding outputs/interpretations in your PDF.

The deduction rules will be similar to those for R.

Grading

The point value for each problem sub-part is indicated. There are 100 points possible on this homework, evaluated based on the correctness of the numerical results and plots. When a problem asks for a plot:

- **Axes and labels:** Always include informative axis labels and, if appropriate, titles.
- **Required elements:** If the instructions ask for a 45° line, smoothing curve, or facets, these must be included.
- **Clarity:** Points may be deducted if plots are cluttered, unreadable, or missing labels.

Submission

Compile your writeup to PDF and submit to Gradescope.

1 R basics (10 points)

1.1 Creating vectors of divisible integers (5 points)

Solution.

```
vec1 = rev(seq(0, 200, 13)) # descend the sequence with step as 13
vec1
```

```
## [1] 195 182 169 156 143 130 117 104 91 78 65 52 39 26 13 0
```

1.2 Creating repeated vectors (5 points)

Solution.

```
# repeat 1 to 25 times given 25 odd numbers between 101 to 149
vec2 = rep(seq(101,149,2), times= seq(1,25))
vec2
```

```
## [1] 101 103 103 105 105 105 107 107 107 107 109 109 109 109 109 111 111 111
## [19] 111 111 111 113 113 113 113 113 113 113 115 115 115 115 115 115 115 115
## [37] 117 117 117 117 117 117 117 117 117 119 119 119 119 119 119 119 119 119
## [55] 119 121 121 121 121 121 121 121 121 121 121 121 123 123 123 123 123 123
## [73] 123 123 123 123 123 123 125 125 125 125 125 125 125 125 125 125 125 125
## [91] 125 127 127 127 127 127 127 127 127 127 127 127 127 127 127 127 129 129 129
## [109] 129 129 129 129 129 129 129 129 129 129 129 129 131 131 131 131 131 131
## [127] 131 131 131 131 131 131 131 131 131 131 133 133 133 133 133 133 133 133
## [145] 133 133 133 133 133 133 133 133 133 133 135 135 135 135 135 135 135 135
## [163] 135 135 135 135 135 135 135 135 135 135 137 137 137 137 137 137 137 137
## [181] 137 137 137 137 137 137 137 137 137 137 139 139 139 139 139 139 139 139
## [199] 139 139 139 139 139 139 139 139 139 139 139 139 141 141 141 141 141 141
## [217] 141 141 141 141 141 141 141 141 141 141 141 141 141 141 141 141 143 143 143
## [235] 143 143 143 143 143 143 143 143 143 143 143 143 143 143 143 143 143 143 143
## [253] 143 145 145 145 145 145 145 145 145 145 145 145 145 145 145 145 145 145 145
## [271] 145 145 145 145 145 145 147 147 147 147 147 147 147 147 147 147 147 147 147
## [289] 147 147 147 147 147 147 147 147 147 147 147 147 147 149 149 149 149 149 149
## [307] 149 149 149 149 149 149 149 149 149 149 149 149 149 149 149 149 149 149 149
## [325] 149
```

Case study: NYC Flights 2013

2 Processing (45 points)

2.1 Import (5 points)

Solution.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
flight_raw = read_csv("NYCflights13sub.csv")
```

```
## Rows: 33672 Columns: 19
## -- Column specification -----
## Delimiter: ","
## chr   (4): carrier, tailnum, origin, dest
## dbl   (14): year, month, day, dep_time, sched_dep_time, dep_delay, arr_time, ...
## dtm   (1): time_hour
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
dim(flight_raw) # It has 33672 rows and 19 columns
```

```
## [1] 33672    19
```

```
flight_raw
```

```
## # A tibble: 33,672 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <dbl> <dbl> <dbl> <dbl>         <dbl>         <dbl>   <dbl>         <dbl>
## 1  2013     1    22   1659           1629          30    2021           1931
## 2  2013     1    22   1743           1745         -2    2107           2120
## 3  2013     1    31   1955           1950          5    2332           2335
## 4  2013     1    29   2012           2020         -8    2241           2245
## 5  2013     1     4   1018            953         25    1119           1110
## 6  2013     1     3    630            633         -3     801            803
## 7  2013     1    30   1649           1650         -1    1852           1826
## 8  2013     1     4   1734           1729          5    2037           2049
```

```
## 9 2013 1 14 1138 1140 -2 1455 1450
## 10 2013 1 6 1456 1500 -4 1655 1656
## # i 33,662 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <dbl>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
# Select numeric/categorical columns
num_col_flight = select(flight_raw, where(is.numeric))
numeric_cols = names(num_col_flight)
cat_cols = setdiff(names(flight_raw), numeric_cols)
print(numeric_cols) # These are numeric columns
```

```
## [1] "year" "month" "day" "dep_time"
## [5] "sched_dep_time" "dep_delay" "arr_time" "sched_arr_time"
## [9] "arr_delay" "flight" "air_time" "distance"
## [13] "hour" "minute"
```

```
print(cat_cols) # These are categorical columns
```

```
## [1] "carrier" "tailnum" "origin" "dest" "time_hour"
```

This Data has 33672 rows and 19 columns. Three numeric variables are: year, month, distance. Three categorical variables are: carrier, origin, dest.

2.2 Tidy: missing values (5 points)

Solution.

```
# Use drop_na, see documentation: https://tidyr.tidyverse.org/reference/drop_na.html
flights_clean = flight_raw |> drop_na(dep_delay, arr_delay, air_time)
print(flights_clean)
```

```
## # A tibble: 32,719 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <dbl> <dbl> <dbl>   <dbl>         <dbl>         <dbl>   <dbl>         <dbl>
## 1 2013     1    22   1659           1629          30    2021           1931
## 2 2013     1    22   1743           1745         -2    2107           2120
## 3 2013     1    31   1955           1950          5    2332           2335
## 4 2013     1    29   2012           2020         -8    2241           2245
## 5 2013     1     4   1018           953         25    1119           1110
## 6 2013     1     3    630           633         -3     801           803
## 7 2013     1    30   1649           1650         -1    1852           1826
## 8 2013     1     4   1734           1729          5    2037           2049
## 9 2013     1    14   1138           1140         -2    1455           1450
## 10 2013     1     6   1456           1500         -4    1655           1656
## # i 32,709 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <dbl>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
# Calculate number of rows removed
num_row_removed = nrow(flight_raw) - nrow(flights_clean)
print(num_row_removed)
```

```
## [1] 953
```

953 rows were dropped due to missingness.

2.3 Tidy: filtering major carriers (15 points)

Solution.

```
# use summarize with n() for carrier counts
carrier_counts = flights_clean |>
  summarize(carrier_counts = n(), .by = carrier) |>
  arrange(desc(carrier_counts))

print(carrier_counts)
```

```
## # A tibble: 16 x 2
##   carrier carrier_counts
##   <chr>         <int>
## 1 UA             5810
## 2 B6             5345
## 3 EV             5106
## 4 DL             4817
## 5 AA             3197
## 6 MQ             2520
## 7 US             1967
## 8 9E             1680
## 9 WN             1217
## 10 VX             515
## 11 FL             305
## 12 AS              80
## 13 F9              70
## 14 YV              59
## 15 HA              29
## 16 OO              2
```

```
# get main carriers with at least 1000 flights
main_carriers = carrier_counts |> filter(carrier_counts >= 1000) |>
  select(carrier)
print(main_carriers$carrier)
```

```
## [1] "UA" "B6" "EV" "DL" "AA" "MQ" "US" "9E" "WN"
```

```
# Further filter flights_clean for main carrier
num_row_pre_filter = nrow(flights_clean)
flights_clean = flights_clean |> filter(carrier %in% main_carriers$carrier)
num_row_post_filter = nrow(flights_clean)
print(flights_clean)
```

```
## # A tibble: 31,659 x 19
##   year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
##   <dbl> <dbl> <dbl>   <dbl>         <dbl>         <dbl>   <dbl>         <dbl>
## 1  2013     1    22    1659             1629          30    2021             1931
## 2  2013     1    22    1743             1745         -2    2107             2120
## 3  2013     1    31    1955             1950          5    2332             2335
## 4  2013     1    29    2012             2020         -8    2241             2245
## 5  2013     1     4    1018              953         25    1119             1110
## 6  2013     1     3     630              633         -3     801              803
## 7  2013     1    30    1649             1650         -1    1852             1826
## 8  2013     1     4    1734             1729          5    2037             2049
## 9  2013     1    14    1138             1140         -2    1455             1450
## 10 2013     1     6    1456             1500         -4    1655             1656
## # i 31,649 more rows
## # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <dbl>,
## #   tailnum <chr>, origin <chr>, dest <chr>, air_time <dbl>, distance <dbl>,
## #   hour <dbl>, minute <dbl>, time_hour <dtm>
```

```
print(num_row_pre_filter - num_row_post_filter)
```

```
## [1] 1060
```

1060 rows were further dropped due to this filtering

2.4 Quality control (20 points)

Solution.

```
# Check min/max value for dep_time
min_max_dep_time = flights_clean |>
  summarize(min_sch_dep = min(sched_dep_time),
            max_sch_dep = max(sched_dep_time),
            min_dep = min(dep_time),
            max_dep = max(dep_time)
  )
print(min_max_dep_time)
```

```
## # A tibble: 1 x 4
##   min_sch_dep max_sch_dep min_dep max_dep
##   <dbl>         <dbl>   <dbl>   <dbl>
## 1       500       2359         1    2400
```

Given our examination above, the first two digits in a four digits number is the hour, and last two digits is the minute for `dep_time` and `sched_dep_time`

```
# Get dep_time and sched_dep_time hours and mins
flights_clean = flights_clean |>
  mutate(
    dep_time_hr = dep_time %/% 100,
    dep_time_min = dep_time %% 100,
    sched_dep_time_hr = sched_dep_time %/% 100,
```

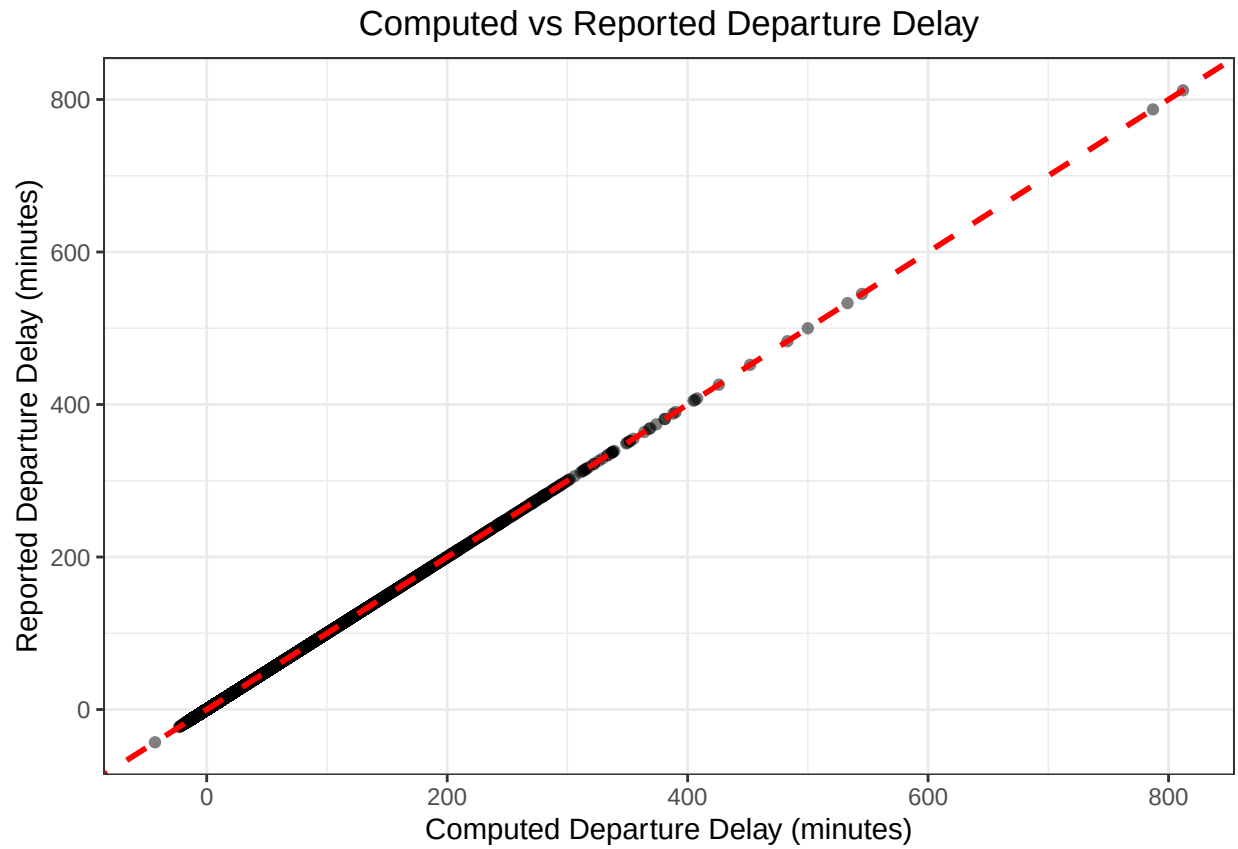
```

    sched_dep_time_min = sched_dep_time %% 100
  )

# get dep_delay_computed
flights_clean = flights_clean |>
  mutate(
    dep_delay_computed = (dep_time_hr - sched_dep_time_hr) * 60 +
      dep_time_min - sched_dep_time_min,
    # assume the actual depart cannot be 10 hours before scheduled depart, we add
    # extra day (1440 min) for day difference
    dep_delay_computed = if_else(
      dep_delay_computed < -600, dep_delay_computed + 1440, dep_delay_computed
    )
  )

# get the ggplot
ggplot(data=flights_clean) +
  geom_point(mapping = aes(x=dep_delay_computed, y=dep_delay), alpha=0.5) +
  geom_abline(slope = 1, intercept = 0,
    color = "red", linetype = "dashed", linewidth = 1) +
  labs(
    title = "Computed vs Reported Departure Delay",
    x = "Computed Departure Delay (minutes)",
    y = "Reported Departure Delay (minutes)"
  ) +
  theme(plot.title = element_text(hjust = 0.5))

```

The computed and provided delay time statistics seem to be the same from the plot and we do not observe any discrepancies

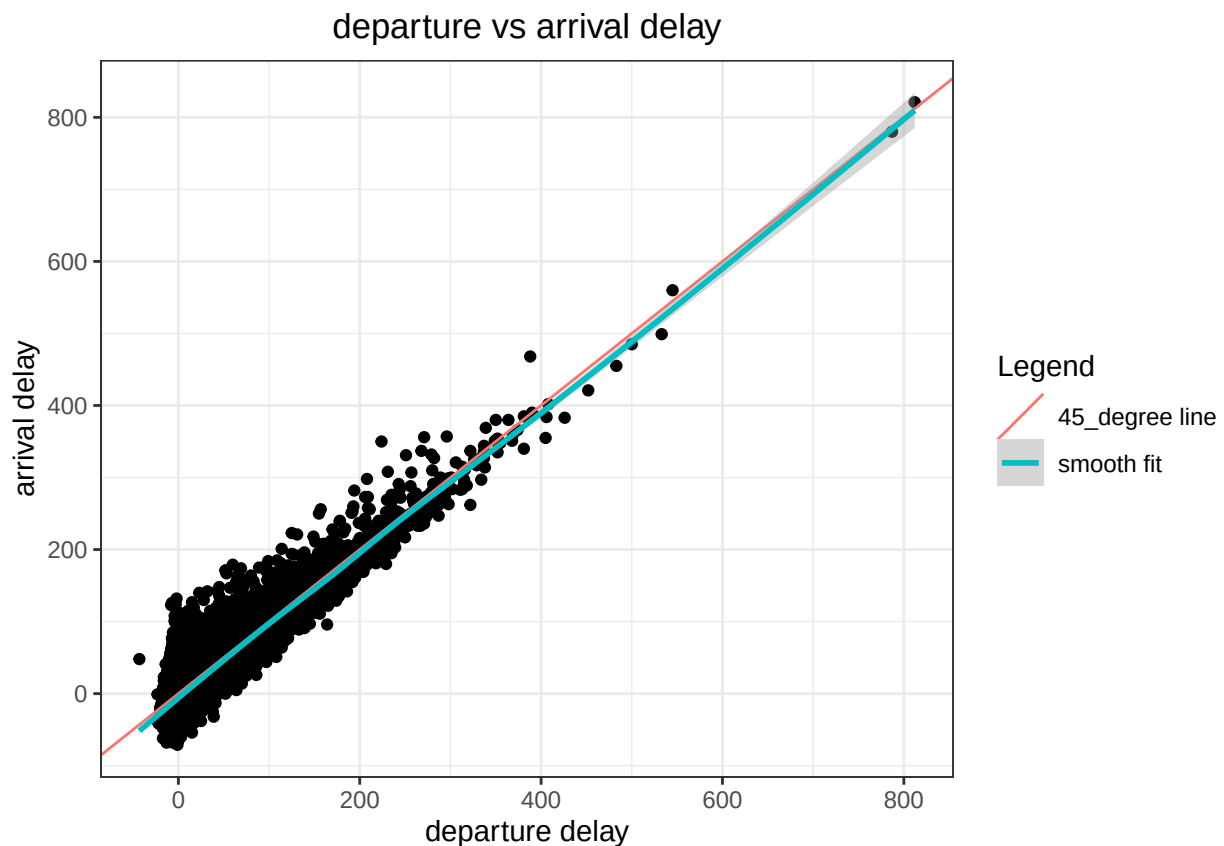
3 Explore (45 points)

3.1 Departure and arrival delays (15 points)

Solution.

```
ggplot(data=flights_clean,
       mapping = aes(x=dep_delay, y=arr_delay)) +
  geom_point() +
  geom_abline(aes(slope = 1, intercept = 0, color="45_degree line")) +
  geom_smooth(aes(color = "smooth fit")) +
  labs(title = "departure vs arrival delay",
       x = "departure delay",
       y = "arrival delay",
       color = "Legend" ) +
  theme(plot.title = element_text(hjust = 0.5))
```

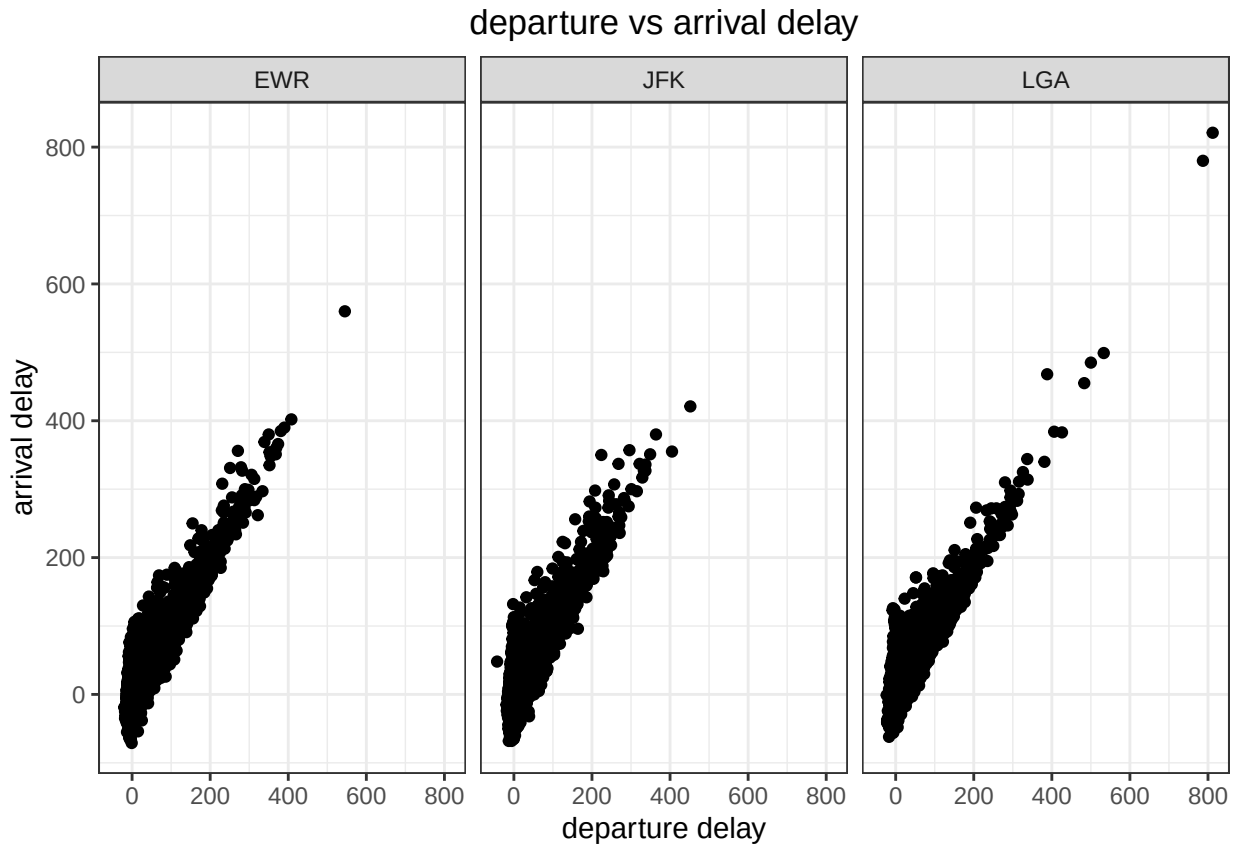
'geom_smooth()' using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



The relationship between departure time and arrival time appears to be linear based on the smooth curve

```
ggplot(data=flights_clean,
       mapping = aes(x=dep_delay, y=arr_delay)) +
  geom_point() +
```

```
labs(title = "departure vs arrival delay",
     x = "departure delay",
     y = "arrival delay") +
theme(plot.title = element_text(hjust = 0.5)) +
facet_wrap(~origin)
```



The departure and arrival delay appears to be linear in all three airports

3.2 Delays by month and by carrier (20 points)

Solution.

```
# Check mean arrival delay time in a month per carrier
delay_aggregate = flights_clean |>
  summarize(delay_month = mean(arr_delay),
            .by = c(month, carrier)
  )
print(delay_aggregate)
```

```
## # A tibble: 108 x 3
##   month carrier delay_month
##   <dbl> <chr>      <dbl>
## 1     1 B6        1.41
## 2     1 AA        3.12
## 3     1 MQ        4.54
```

```
## 4      1 EV      22.0
## 5      1 UA       3.87
## 6      1 DL      -6.20
## 7      1 US      -3.68
## 8      1 WN       5.52
## 9      1 9E      15.1
## 10     2 US      -0.463
## # i 98 more rows
```

```
# Check mean arrival delay time in a year per carrier
delay_yr = flights_clean |>
  summarize(delay_by_year = mean(arr_delay),
            .by = carrier # since all data are in year 2013
  )
print(delay_yr)
```

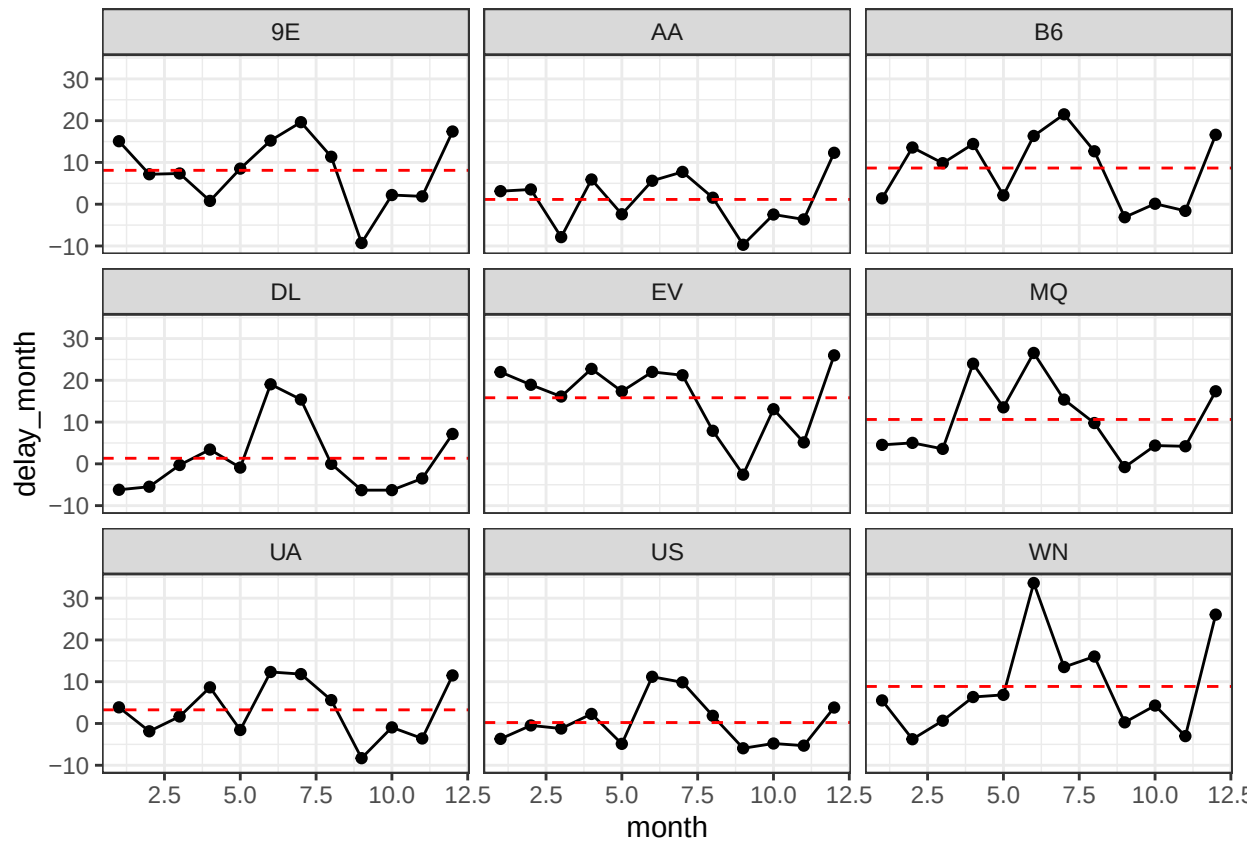
```
## # A tibble: 9 x 2
##   carrier delay_by_year
##   <chr>         <dbl>
## 1 B6             9.08
## 2 AA             0.991
## 3 MQ            10.8
## 4 EV            15.5
## 5 UA             3.41
## 6 DL             1.64
## 7 US             0.362
## 8 WN             9.13
## 9 9E             7.91
```

```
# Join delay_yr to delay_aggregate
delay_aggregate = left_join(delay_aggregate, delay_yr, by="carrier")
print(delay_aggregate)
```

```
## # A tibble: 108 x 4
##   month carrier delay_month delay_by_year
##   <dbl> <chr>         <dbl>         <dbl>
## 1      1 B6          1.41           9.08
## 2      1 AA          3.12           0.991
## 3      1 MQ          4.54          10.8
## 4      1 EV         22.0          15.5
## 5      1 UA          3.87           3.41
## 6      1 DL        -6.20           1.64
## 7      1 US        -3.68           0.362
## 8      1 WN          5.52           9.13
## 9      1 9E         15.1           7.91
## 10     2 US        -0.463          0.362
## # i 98 more rows
```

```
# Plot the monthly mean arrive time
# compute avg carrier delay
carrier_means <- delay_aggregate |>
  summarize(mean_delay = mean(delay_month), .by = carrier)
```

```
ggplot(data = delay_aggregate, aes(x = month, y = delay_month)) +
  geom_point() +
  geom_line() +
  facet_wrap(~carrier) +
  geom_hline(data = carrier_means, aes(yintercept = mean_delay), color = "red", linetype = "dashed")
```



Carrier efficiency (10 points)

Solution.

```
# Get efficiency per carrier
efficiency_carrier = flights_clean |>
  summarize(on_time_rate = sum(arr_delay <= 0) / n(),
            mean_dep_delay = mean(
              ifelse(dep_delay > 0, dep_delay, 0)
            ), .by = carrier) |>
  mutate(efficiency = on_time_rate / (1 + mean_dep_delay))

print(efficiency_carrier)
```

```
## # A tibble: 9 x 4
##   carrier on_time_rate mean_dep_delay efficiency
##   <chr>      <dbl>      <dbl>      <dbl>
## 1 B6         0.567        15.5      0.0344
## 2 AA         0.653        12.2      0.0494
```

```
## 3 MQ          0.528      14.3      0.0345
## 4 EV          0.523      22.0      0.0228
## 5 UA          0.612      13.7      0.0417
## 6 DL          0.651      12.0      0.0503
## 7 US          0.643       6.98     0.0806
## 8 WN          0.557      18.4      0.0288
## 9 9E          0.602      19.6      0.0292
```

```
# Get top three efficient carriers
print(arrange(efficiency_carrier, desc(efficiency)))
```

```
## # A tibble: 9 x 4
##   carrier on_time_rate mean_dep_delay efficiency
##   <chr>      <dbl>      <dbl>      <dbl>
## 1 US          0.643       6.98     0.0806
## 2 DL          0.651      12.0     0.0503
## 3 AA          0.653      12.2     0.0494
## 4 UA          0.612      13.7     0.0417
## 5 MQ          0.528      14.3     0.0345
## 6 B6          0.567      15.5     0.0344
## 7 9E          0.602      19.6     0.0292
## 8 WN          0.557      18.4     0.0288
## 9 EV          0.523      22.0     0.0228
```

Based on above tibble US, DL, AA are top 3 carriers with highest efficiency